Contracts to Effects Compiler Correctness

(Shallow Handler)

Overview

Proof for compiler from Contracts to Untyped Shallow Effects, the handler semantic is shallow, where the continuation does not reinstall the handler.

The only rule that changes from the deep semantics is Step-E-Perform, the continuation k no longer wraps E with $\texttt{handle}\ h\ \cdot$.

$$\frac{\mathsf{op} \notin \texttt{bop}(E) \wedge (\mathsf{op} \to f) \in h \qquad k = \lambda x.E[x]}{\texttt{handle}\ h\ E[\texttt{perform}\ \mathsf{op}\ v] \ \rightarrow\ f\ v\ k}\ \text{Step-E-Perform-Shallow}$$

$$\begin{array}{lcl} \texttt{h}_{\mathscr{C}} : \texttt{lab} & = & \{\mathscr{C} \to \forall\alpha.\langle \texttt{blab}, \langle \alpha \to \mathsf{bool}, \alpha\rangle\rangle \to \alpha\} \\ \mathsf{H} & = & \mu h.\ \texttt{handler}\ \{\mathscr{C} \to \lambda\langle l, \langle e, v\rangle\rangle.\lambda k.h\ (\lambda_.\ \text{if}\ e\ v\ \text{then}\ k\ v\ \text{else}\ \texttt{error}_l)\} \\ \mathsf{wrap} & = & \Lambda\alpha.\ \lambda f : \mathsf{unit} \to \alpha.\ \mathsf{H}\ f \end{array}$$

where $\lambda\langle l, \langle e, v\rangle\rangle$ deconstructs the argument as tuple pattern

The handler clause re-installs H around the rest of the program by itself, instead of the continuation doing it. So the whole program always runs under exactly one $\texttt{handle}\ \texttt{h}_{\mathscr{C}}\ \cdot$ at the top, and the predicate of a check runs under the same handler as the program. We write $\texttt{handle}\ \texttt{h}_{\mathscr{C}}\ e$ for the program running under the handler of H.

Compared to the deep semantics, there are no states without handle, and there is no nested handle for predicates. This makes the relation much simpler.

Compiler Correctness by proving (1) Contracts $\sim$ Untyped Effects, (2) Untyped Effects $\sim$ Effects, then obtain Contracts $\sim$ Effects. (2) is trivial.

In this proof, we use e^* as shorthand to denote e untyped.

Assumptions:

- Contract predicates are values ($\mathsf{flat}(e)$ with $e = \lambda x.e_0$) and pure (they do not use the store).
- Tuple contracts and mutable cells are not covered.

Relation

Let's first define these relations:

- $R_e \subseteq$ Contracts $e \times$ Untyped `Effects` e

  If $(e, e') \in R_e$, we write $e \sim_{R_e} e'$ or $e \sim e'$. We define $\tilde{e}$ as shorthand for any expression $\tilde{e}$ satisfying $e \sim \tilde{e}$.

- $R_E \subseteq$ Contracts $E \times$ Untyped `Effects` E

  If $(E, E') \in R_E$, we write $E \sim_{R_E} E'$ or $E \sim E'$. We define $\tilde{E}$ as shorthand for any context $\tilde{E}$ satisfying $E \sim \tilde{E}$.

In the shallow semantics, the innermost check captures everything outside of it as its continuation, so a check is related together with its whole context:

$E\left[\mathsf{check}_j^k(e, v)\right] \sim \text{if } \tilde{e} \text{ then } \left(\lambda x.\widetilde{E[x]}\right) \tilde{v} \text{ else } \mathtt{error}_k$

where e has no check in evaluation position. E may contain other checks, $\widetilde{E[x]}$ is again related by this rule, and it has one less check.

$$\mathrm{EQ}(e, e') = \begin{cases} \text{true if } e' = \mathtt{handle}\ \mathtt{h}_{\mathscr{C}}\ \tilde{e} \\ \text{true if } e = E\left[\mathsf{blame}_j^k\right] \text{ and } e' = \mathtt{handle}\ \mathtt{h}_{\mathscr{C}}\ \mathtt{error}_k \\ \text{false otherwise} \end{cases}$$

$\boxed{\sim \subseteq \text{Contracts } E \times \text{Untyped } \mathtt{Effects}\ E}$

Define $\tilde{E}$ such that $E \sim \tilde{E}$. If E contains a check, only the last row applies, on the innermost check.

$$\begin{aligned}
\square &\sim \square \\
E\ e &\sim \tilde{E}\ \tilde{e} \\
v\ E &\sim \tilde{v}\ \tilde{E} \\
\mathsf{mon}_j^{k,l}(\mathsf{flat}(e), E) &\sim \mathtt{perform}\ \mathscr{C}\ \langle k, \langle \tilde{e}, \tilde{E} \rangle \rangle \\
\mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, E) &\sim \tilde{e}_1\ \tilde{E} \\
&\quad e_1 \text{ as in the } \mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, e) \text{ row below} \\
\mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, E) &\sim \tilde{e}_1\ \tilde{E} \\
&\quad e_1 \text{ as in the } \mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, e) \text{ row below}
\end{aligned}$$

$$E\left[\mathsf{check}_j^k(E_1, v)\right] \sim \text{if } \tilde{E}_1 \text{ then } \left(\lambda x.\widetilde{E[x]}\right) \tilde{v} \text{ else } \mathtt{error}_k$$
where E_1 has no check

The other contexts ($\langle E, e\rangle$, $o(v..., E, e...)$, if E then e_1 else e_2, ...) are related homomorphically.

$\sim \subseteq$ Contracts $e \times$ Untyped $\mathtt{Effects}$ e

$$\begin{aligned}
v &\sim \tilde{v} \\
e_1\, e_2 &\sim \tilde{e}_1\, \tilde{e}_2 \\
\mathsf{mon}_j^{k,l}(\mathsf{flat}(e_1), e_2) &\sim \mathtt{perform}\ \mathscr{C}\ \langle k, \langle \tilde{e}_1, \tilde{e}_2\rangle\rangle \\
\mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, e) &\sim \tilde{e}_1\, \tilde{e} \\
&\quad e_1 = \lambda f.\lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right) \\
\mathsf{mon}_j^{k,l}(\kappa_1 \to_i &\sim \tilde{e}_1\, \tilde{e} \\
\lambda y.\kappa_2, e) & \\
&\quad e_1 = \lambda f.\lambda x.\left(\lambda y.\mathsf{mon}_j^{k,l}\left(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right)\right) \mathsf{mon}_j^{l,j}(\kappa_1, x) \\
E\left[\mathsf{check}_j^k(e, v)\right] &\sim \text{if } \tilde{e} \text{ then } \left(\lambda x.\widetilde{E[x]}\right) \tilde{v} \text{ else } \mathtt{error}_k \\
&\quad \text{where } e \text{ has no check in evaluation position (innermost check)} \\
e\left[y := e_y\right] &\sim \tilde{e}\left[y := \tilde{v}_y\right] \\
&\quad \text{where } e_y = \mathsf{mon}_j^{l,j}(\kappa_1, v),\ e_y \twoheadrightarrow^* v_y \text{ and } v_y \sim \tilde{v}_y \text{ (indy argument)} \\
\mathsf{blame}_j^k &\sim \mathtt{error}_k
\end{aligned}$$

The other expressions ($\langle e_1, e_2\rangle$, $o(e...)$, if e_1 then e_2 else e_3, $\mu x.e$, ...) are related homomorphically.

$\sim \subseteq$ Contracts $v \times$ Untyped $\mathtt{Effects}$ v

$$\begin{aligned}
b &\sim b \\
N &\sim N \\
\lambda x.e &\sim \lambda x.\tilde{e} \\
\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v) &\sim \tilde{e} \\
&\quad e = \lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_2, \tilde{v}\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right)
\end{aligned}$$

$$\mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v) \quad \sim \quad \tilde{e}$$
$$e = \lambda x.\left(\lambda y.\mathsf{mon}_j^{k,l}\left(\kappa_2, \tilde{v}\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right)\right)\ \mathsf{mon}_j^{l,j}(\kappa_1, x)$$

Lemma 0.1. (Relation is correct) If $E \sim \tilde{E}$, $e \sim \tilde{e}$ and e has no check in evaluation position, then $E[e] \sim \tilde{E}[\tilde{e}]$.

Proof. By induction on E.

- Case $E = \square$

$E[e] = e$	Context Plug in
$\tilde{E} = \square$	Definition
$\tilde{E}[\tilde{e}] = \tilde{e}$	Context Plug in
$E[e] \sim \tilde{E}[\tilde{e}]$	Given

- Case $E = E_0\ e_0$

$E_0[e] \sim \widetilde{E_0}[\tilde{e}]$	I.H.
$E[e] = E_0[e]\ e_0$	Context Plug in
$\tilde{E}[\tilde{e}] = \widetilde{E_0}[\tilde{e}]\ \tilde{e}_0$	Definition
$E[e] \sim \tilde{E}[\tilde{e}]$	Definition

- Case $E = v\ E_0$

$E_0[e] \sim \widetilde{E_0}[\tilde{e}]$	I.H.
$E[e] = v\ E_0[e]$	Context Plug in
$\tilde{E}[\tilde{e}] = \tilde{v}\ \widetilde{E_0}[\tilde{e}]$	Definition
$E[e] \sim \tilde{E}[\tilde{e}]$	Definition

- Case $E = E_0\left[\mathsf{check}_j^k(E_1, v)\right]$, where E_1 has no check in evaluation position

$E_1[e] \sim \widetilde{E_1}[\tilde{e}]$	I.H.
$E[e] = E_0\left[\mathsf{check}_j^k(E_1[e], v)\right]$	Context Plug in
$E[e] \sim \text{if } \widetilde{E_1}[\tilde{e}] \text{ then } \left(\lambda x.\widetilde{E_0[x]}\right)\ \tilde{v} \text{ else } \mathtt{error}_k$	Check row
$\text{if } \widetilde{E_1}[\tilde{e}] \text{ then } \left(\lambda x.\widetilde{E_0[x]}\right)\ \tilde{v} \text{ else } \mathtt{error}_k = \tilde{E}[\tilde{e}]$	Definition
$E[e] \sim \tilde{E}[\tilde{e}]$	

- Other context forms follow similarly.

□

Lemma 0.2. (Relation is closed under substitution) If $e \sim \tilde{e}$ and $v \sim \tilde{v}$ then $e[x := v] \sim \tilde{e}[x := \tilde{v}]$.

Proof. By induction on $e \sim \tilde{e}$. Every row is homomorphic on x, and the bound variables f, x, y in the rows are chosen fresh. □

Lemma 0.3. (Compiler is subset of relation)

If $\cdot \vdash e_1 : \tau \rightarrowtail e_2$ then $e_1 \sim e_2^*$.

Proof.

By induction on structure of $\cdot \vdash e_1 : \tau \rightarrowtail e_2$.

- Case

$$\frac{\cdot \vdash e_3 : \tau \to \mathsf{bool} \rightarrowtail e_5 \qquad \cdot \vdash e_4 : \tau \rightarrowtail e_6}{\cdot \vdash \mathsf{mon}_j^{k,l}(\mathsf{flat}(e_3), e_4) : \tau \rightarrowtail \mathtt{perform}\ \mathscr{C}\ \tau\ \langle k, \langle e_5, e_6 \rangle\rangle}$$

$e_2^* = \mathtt{perform}\ \mathscr{C}\ \langle k, \langle e_5^*, e_6^* \rangle\rangle$

By I.H. $e_3 \sim e_5^*$ and $e_4 \sim e_6^*$.

Then $e_1 \sim e_2^*$ by relation.

- Case

$$\frac{\cdot \vdash e_3 : \tau_1 \to \tau_2 \rightarrowtail e_4}{\cdot \vdash \lambda f.\lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right) : (\tau_1 \to \tau_2) \to (\tau_1 \to \tau_2) \rightarrowtail e_5 \qquad \cdot \vdash \mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, e_3) : \tau_1 \to \tau_2 \rightarrowtail e_5\ e_4}$$

By I.H., $e_3 \sim e_4^*$

By I.H., $\lambda f.\lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right) \sim e_5^*$

Then $e_1 \sim e_2^*$ by relation.

- Case

$$\frac{\cdot \vdash e_3 : \tau_1 \to \tau_2 \rightarrowtail e_4}{\cdot \vdash \lambda f.\lambda x.\left(\lambda y.\mathsf{mon}_j^{k,l}\left(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right)\right)\ \mathsf{mon}_j^{l,j}(\kappa_1, x) : (\tau_1 \to \tau_2) \to (\tau_1 \to \tau_2) \rightarrowtail e_5 \qquad \cdot \vdash \mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, e_3) : \tau_1 \to \tau_2 \rightarrowtail e_5\ e_4}$$

By I.H., $e_3 \sim e_4^*$

By I.H., $\lambda f.\lambda x.\left(\lambda y.\mathsf{mon}_j^{k,l}\left(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right)\right)\ \mathsf{mon}_j^{l,j}(\kappa_1, x) \sim e_5^*$

Then $e_1 \sim e_2^*$ by relation.

- Other cases are homomorphic, by I.H.

□

Compiler Correctness

Theorem 0.4. If $\cdot \vdash e : \tau \rightarrowtail e'$ and $e^* = \text{untyped}(\mathsf{wrap}[\tau]\ (\lambda_.e'))$ and $e \twoheadrightarrow^* r$ then $e^* \Longrightarrow^* \tilde{r}$.

Where r is a value v or blame_j^k.

Proof.

$e^* \Longrightarrow^* e_1'$ and $\text{EQ}(e, e_1')$ by Lemma 0.5.

$e_1' \Longrightarrow^* \tilde{r}$ by Lemma 0.6.

$e^* \Longrightarrow^* \tilde{r}$ by combining steps. □

Lemma 0.5. (Wrapped compiled program steps to an equivalent state with source program)

If $\cdot \vdash e : \tau \rightarrowtail e'$ and $e_1^* = \text{untyped}(\mathsf{wrap}[\tau]\ (\lambda_.e'))$ then it exists e_2^* such that $e_1^* \Longrightarrow^* e_2^*$ and $\text{EQ}(e, e_2^*)$.

Proof.

By Lemma 0.3, $e \sim e'^*$.

$e_1^* \Longrightarrow^* \mathsf{H}\ \lambda_.e'^* \Longrightarrow^* \texttt{handler}\ \mathtt{h}_{\mathscr{C}}\ \lambda_.e'^* \Longrightarrow^* \texttt{handle}\ \mathtt{h}_{\mathscr{C}}\ e'^* = e_2^*$

By definition, $\text{EQ}(e, e_2^*)$ (case 1). □

Lemma 0.6. If $e \twoheadrightarrow^* r$ and $\text{EQ}(e, e')$ then $e' \Longrightarrow^* \tilde{r}$.

Proof. By breaking down the $\twoheadrightarrow^*$, and induction on it.

- Case $r \twoheadrightarrow^* r$ with $r = v$

  $e' = \texttt{handle}\ \mathtt{h}_{\mathscr{C}}\ \tilde{v}$ because $\text{EQ}(v, e')$ (case 2 is invalid).

  $e' \blacktriangleright \tilde{v}$ by Step-E-Return.

- Case $r \twoheadrightarrow^* r$ with $r = \mathsf{blame}_j^k$

  $e' = \texttt{handle}\ \mathtt{h}_{\mathscr{C}}\ \texttt{error}_k$ in both cases of `treq`.

  $e' \Longrightarrow \texttt{error}_k$ by Step-E-Error.

- Case $e \twoheadrightarrow e_1$ and $e_1 \twoheadrightarrow^* r$

  By simulation, we have $e' \Longrightarrow^* e_1'$ such that $\text{EQ}(e_1, e_1')$.

  By IH, $e_1' \Longrightarrow^* \tilde{r}$.

  By chaining steps, $e' \Longrightarrow^* \tilde{r}$.

□

We also need to make a claim that in no where in $\tilde{e}$ do we have `handle` or $\texttt{perform}\ \texttt{eff}$ that $\texttt{eff} \neq \mathscr{C}$, so $\mathscr{C} \notin \texttt{bop}(E')$ for every target context E' under the top handle.

Lemma 0.7. (Performing a check) If E' has no `handle`, then

$$\texttt{handle}\ \texttt{h}_{\mathscr{C}}\ E'[\texttt{perform}\ \mathscr{C}\ \langle k, \langle \tilde{e}, \tilde{v} \rangle\rangle]$$

$$\Longrightarrow^{*} \texttt{handle}\ \texttt{h}_{\mathscr{C}}\ \ \text{if}\ \tilde{e}\ \tilde{v}\ \text{then}\ (\lambda x.E'[x])\ \tilde{v}\ \text{else}\ \texttt{error}_k$$

Proof.

$\texttt{handle}\ \texttt{h}_{\mathscr{C}}\ E'[\texttt{perform}\ \mathscr{C}\ \langle k, \langle \tilde{e}, \tilde{v} \rangle\rangle]$

$\rightarrow (\lambda\langle l, \langle e, v\rangle\rangle.\lambda k.\ \mathsf{H}\ (\lambda_.\ \text{if}\ e\ v\ \text{then}\ k\ v\ \text{else}\ \texttt{error}_l))\ \langle k, \langle \tilde{e}, \tilde{v}\rangle\rangle\ (\lambda x.E'[x])$ (Step-E-Perform-Shallow)

$\Longrightarrow^{*} \mathsf{H}\ (\lambda_.\ \text{if}\ \tilde{e}\ \tilde{v}\ \text{then}\ (\lambda x.E'[x])\ \tilde{v}\ \text{else}\ \texttt{error}_k)$

$\Longrightarrow^{*} \texttt{handle}\ \texttt{h}_{\mathscr{C}}\ \ \text{if}\ \tilde{e}\ \tilde{v}\ \text{then}\ (\lambda x.E'[x])\ \tilde{v}\ \text{else}\ \texttt{error}_k$ (unfold μ, Step-E-Handler, β)

□

Lemma 0.8. If $e_1 \twoheadrightarrow e_2$ and $\mathrm{EQ}(e_1, e_1')$ then there exists e_2' such that $e_1' \Longrightarrow^{*} e_2'$ and $\mathrm{EQ}(e_2, e_2')$.

Proof of Lemma 0.8. By case analysis on $e_1 \twoheadrightarrow e_2$ with respect to $\mathrm{EQ}(e_1, e_1')$.

- Case $e_1 = E\left[\mathsf{blame}_j^k\right] \twoheadrightarrow \mathsf{blame}_j^k = e_2$ (Step-C-Blame)
  - Case $\mathrm{EQ}(e_1, e_1')$ is case 2, $e_1' = \texttt{handle}\ \texttt{h}_{\mathscr{C}}\ \texttt{error}_k$.

    $e_2' = e_1'$, and $\mathrm{EQ}\left(\mathsf{blame}_j^k, e_2'\right)$ (case 2 with $E = \square$).
  - Case $\mathrm{EQ}(e_1, e_1')$ is case 1, $e_1' = \texttt{handle}\ \texttt{h}_{\mathscr{C}}\ \tilde{E}[\texttt{error}_k]$.

    $e_1' \Longrightarrow \texttt{handle}\ \texttt{h}_{\mathscr{C}}\ \texttt{error}_k = e_2'$ by Step-E-Error.

    Obtain $\mathrm{EQ}\left(\mathsf{blame}_j^k, e_2'\right)$ (case 2 with $E = \square$).

Otherwise, e_1 is not blamed and $\mathrm{EQ}(e_1, e_1')$ is case 1. By decomposition lemma, $e_1 = E[e_3]$, $e_2 = E[e_4]$ and that $e_3 \dashrightarrow e_4$ is a redex reduction.

If e_3 is not a check, $e_1' = \texttt{handle}\ \texttt{h}_{\mathscr{C}}\ \tilde{E}[\tilde{e}_3]$ by Lemma 0.1.

Note that $\tilde{E}$ is an evaluation context of `Effects` without `handle`. Unlike the deep proof, there is no case split on the checks in E, the redex steps under the same top handle.

- Case $e_3 = \mathsf{mon}_j^{k,l}(\mathsf{flat}(e), v) \dashrightarrow \mathsf{check}_j^k(e\ v, v) = e_4$

  $\tilde{e}_3 = \mathtt{perform}\ \mathscr{C}\ \langle k, \langle \tilde{e}, \tilde{v} \rangle \rangle$

  $e_1' \Longrightarrow^* \mathtt{handle}\ \mathtt{h}_{\mathscr{C}}$ if $\tilde{e}\ \tilde{v}$ then $\left(\lambda x.\tilde{E}[x]\right)\ \tilde{v}$ else $\mathtt{error}_k = e_2'$ by Lemma 0.7.

  $e_2 = E\left[\mathsf{check}_j^k(e\ v, v)\right]$, and $e\ v$ has no check in evaluation position.

  $e_2 \sim$ if $\tilde{e}\ \tilde{v}$ then $\left(\lambda x.\widetilde{E[x]}\right)\ \tilde{v}$ else $\mathtt{error}_k$ by the check row.

  $\widetilde{E[x]} = \tilde{E}[x]$ by Lemma 0.1.

  Obtain $\mathrm{EQ}(e_2, e_2')$ (case 1).

- Case $e_3 = \mathsf{check}_j^k(\mathsf{true}, v) \dashrightarrow v = e_4$

  $e_1 = E\left[\mathsf{check}_j^k(\mathsf{true}, v)\right]$, the check is the innermost one.

  $e_1' = \mathtt{handle}\ \mathtt{h}_{\mathscr{C}}$ if true then $\left(\lambda x.\widetilde{E[x]}\right)\ \tilde{v}$ else $\mathtt{error}_k$ by the check row.

  $e_1' \Longrightarrow^* \mathtt{handle}\ \mathtt{h}_{\mathscr{C}}\ \widetilde{E[x]}[x := \tilde{v}] = e_2'$

  $e_2 = E[v] = E[x][x := v] \sim \widetilde{E[x]}[x := \tilde{v}]$ by Lemma 0.2.

  Obtain $\mathrm{EQ}(e_2, e_2')$ (case 1).

- Case $e_3 = \mathsf{check}_j^k(\mathsf{false}, v) \dashrightarrow \mathsf{blame}_j^k = e_4$

  $e_1 = E\left[\mathsf{check}_j^k(\mathsf{false}, v)\right]$, the check is the innermost one.

  $e_1' = \mathtt{handle}\ \mathtt{h}_{\mathscr{C}}$ if false then $\left(\lambda x.\widetilde{E[x]}\right)\ \tilde{v}$ else $\mathtt{error}_k$ by the check row.

  $e_1' \Longrightarrow \mathtt{handle}\ \mathtt{h}_{\mathscr{C}}\ \mathtt{error}_k = e_2'$

  $e_2 = E\left[\mathsf{blame}_j^k\right]$

  Obtain $\mathrm{EQ}(e_2, e_2')$ (case 2).

- Case $e_3 = \mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, v) \dashrightarrow \mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v) = e_4$

  Let $e_5 = \lambda f.\lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right) = \lambda f.\lambda x.e_{5a}$

  $\tilde{e}_3 = \tilde{e}_5\ \tilde{v}$

  $e_1' \Longrightarrow \mathtt{handle}\ \mathtt{h}_{\mathscr{C}}\ \tilde{E}[\lambda x.\tilde{e}_{5a}[f := \tilde{v}]] = e_2'$ by β.

  $\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v) \sim \lambda x.\tilde{e}_{5a}[f := \tilde{v}]$ by the relation on values.

  Obtain $\mathrm{EQ}(e_2, e_2')$ (case 1) by Lemma 0.1.

- Case $e_3 = \mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v) \dashrightarrow \mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v) = e_4$

  Same as the case $\mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, v)$, with

  $e_5 = \lambda f.\lambda x.\left(\lambda y.\mathsf{mon}_j^{k,l}\left(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right)\right)\ \mathsf{mon}_j^{l,j}(\kappa_1, x) = \lambda f.\lambda x.e_{5a}$

- Case $e_3 = \mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v_1)\ v_2 \dashrightarrow \mathsf{mon}_j^{k,l}\left(\kappa_2, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\right) = e_4$

  $\tilde{e}_3 = \tilde{e}_5\ \tilde{v}_2$ where $e_5 = \lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_2, \tilde{v}_1\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right)$

  $e_1' \Longrightarrow \mathtt{handle}\ \mathtt{h}_{\mathscr{C}}\ \tilde{E}[\tilde{e}_4] = e_2'$ by β and Lemma 0.2.

  Obtain $\mathrm{EQ}(e_2, e_2')$ (case 1) by Lemma 0.1.

- Case $e_3 = \mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v_1)\ v_2 \dashrightarrow \mathsf{mon}_j^{k,l}\left(\kappa_3, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\right) = e_4$ where $\kappa_3 = \kappa_2\left[y := \mathsf{mon}_j^{l,j}(\kappa_1, v_2)\right]$

  $\tilde{e}_3 = \tilde{e}_5\ \tilde{v}_2$ where $e_5 = \lambda x.(\lambda y.e_{5a})\ \mathsf{mon}_j^{l,j}(\kappa_1, x)$ and $e_{5a} = \mathsf{mon}_j^{k,l}\left(\kappa_2, \tilde{v}_1\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right)$

  $e_1' \Longrightarrow \mathtt{handle}\ \mathtt{h}_{\mathscr{C}}\ \tilde{E}[(\lambda y.\tilde{e}_{5a}[x := \tilde{v}_2])\ \tilde{e}_7]$ by β, where $e_7 = \mathsf{mon}_j^{l,j}(\kappa_1, v_2)$

  The target checks the indy argument $\mathsf{mon}_j^{l,j}(\kappa_1, v_2)$ first, the source checks the argument $\mathsf{mon}_j^{l,k}(\kappa_1, v_2)$ first. For $\kappa_1 = \mathsf{flat}(e_p)$ both become $\mathsf{check}_j^l(e_p\ v_2, v_2)$, so we pair the two checks.

  - Case the check fails.

    Both programs reach $\mathsf{check}_j^l(\mathsf{false}, v_2)$, then blame_j^l and $\mathtt{handle}\ \mathtt{h}_{\mathscr{C}}\ \mathtt{error}_l$ as in the case $\mathsf{check}_j^k(\mathsf{false}, v)$.

  - Case the check succeeds, $\mathsf{mon}_j^{l,j}(\kappa_1, v_2) \twoheadrightarrow^* v_y$ ($v_y = v_2$ for flat κ_1, otherwise $v_y = \mathsf{guard}_j^{l,j}(\kappa_1, v_2)$).

    Let $e_6 = \mathsf{mon}_j^{k,l}\left(\kappa_2, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\right)$, then $e_2 = E\left[e_6\left[y := \mathsf{mon}_j^{l,j}(\kappa_1, v_2)\right]\right]$.

    $e_1' \Longrightarrow^* \mathtt{handle}\ \mathtt{h}_{\mathscr{C}}\ \tilde{E}\left[\tilde{e}_6\left[y := \tilde{v}_y\right]\right] = e_2'$

    By purity, the target's argument check $\mathsf{mon}_j^{l,k}(\kappa_1, \tilde{v}_2)$ succeeds again. Each later evaluation of $\mathsf{mon}_j^{l,j}(\kappa_1, v_2)$ inside κ_2 in the source gives v_y again and is matched by zero target steps.

    $e_6\left[y := \mathsf{mon}_j^{l,j}(\kappa_1, v_2)\right] \sim \tilde{e}_6\left[y := \tilde{v}_y\right]$ by the indy row.

    Obtain $\mathrm{EQ}(e_2, e_2')$ (case 1).

- Case e_3 is a base reduction (β, μ, `injl`, `injr`, `ife`, o)

  $\tilde{e}_3$ is the same redex in `Effects`, and $\tilde{e}_3 \rightarrow \tilde{e}_4$ by the same rule, using Lemma 0.2 for β and μ.

  $e_1' \Longrightarrow \mathtt{handle}\ \mathtt{h}_{\mathscr{C}}\ \tilde{E}[\tilde{e}_4] = e_2'$

  Obtain $\mathrm{EQ}(e_2, e_2')$ (case 1) by Lemma 0.1.

□